

s10 System Prompt / Memory / Skills 学习笔记

基于本次对话、你的 s08/s09/s10 代码与运行日志整理

重点：本笔记的主线是：s08 解决 messages 太长；s09 解决长期 memory；s10 解决 system prompt 动态组装与缓存。你的综合版需要把三者合体，但不能让职责混在一起。

目录

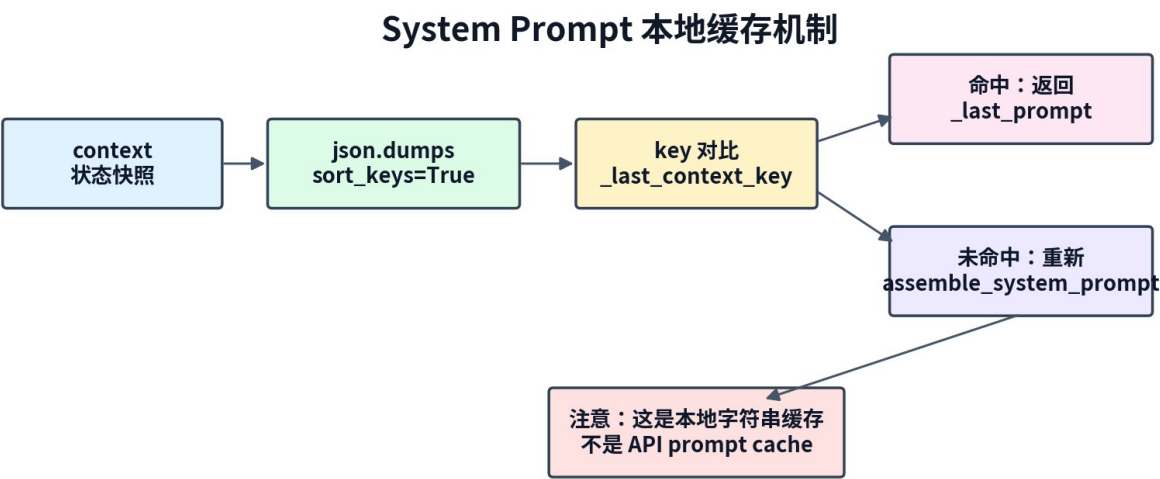
- 1. 本次对话脉络总览
- 2. get_system_prompt 与本地缓存
- 3. context / content / messages 的关系
- 4. MEMORY.md 与完整 memory 全文注入
- 5. s08 / s09 / s10 参考答案的职责边界
- 6. 你的老代码对应问题与优化原因
- 7. Windows 编码报错复盘
- 8. Skills 注入机制
- 9. 最终推荐架构与检查清单

1. 本次对话脉络总览

阶段	你提出的问题	核心结论
system prompt 缓存	逐行解释 get_system_prompt(context)	用 json.dumps(context) 生成稳定 key; context 没变就返回 _last_prompt。
本地缓存 vs 模型缓存	缓存命中和模型自身缓存有什么区别	本地缓存只省字符串拼装; API prompt cache 省模型服务端重复处理前缀 token。
content 来源	content 是哪来的	content 是 messages 中每条消息的正文; context 是系统提示词的运行状态包。
memory 生命周期	MEMORY.md 里的字符串怎么交给模型	MEMORY.md/index 被读入 context; 完整 memory 需要 load_memories() 检索并读取文件全文。
代码检查	这版程序对吗	能跑不等于架构正确; 重点问题是重复注入、compact 协议风险、subagent results 隐患。
参考答案对照	s10 是否与 memory 解耦	s10 只演示 context->system; s09 才有相关 memory 全文注入。
运行日志排错	写入 MEMORY.md 后 UnicodeDecodeError	Windows 默认 GBK 读取 UTF-8 文件; 应统一 encoding="utf-8"。
skills 注入	当前 skills 注入怎么实现	system 默认注入 skill catalog; 完整 skill 由 load_skill 按需返回。

重点：本次最重要的转变：不要把 memory 偷塞进 user.content; 应该统一通过 context 进入 system prompt。

2. get_system_prompt 与本地缓存



重点：get_system_prompt(context) 的缓存对象是“已经组装好的 system prompt 字符串”，不是模型服务端的 prompt cache。

对比项	本地 get_system_prompt 缓存	模型/API prompt cache
发生位置	本地 Python 进程	模型服务端
缓存对象	_last_prompt 字符串	prompt 前缀 token 的处理结果
判断依据	json.dumps(context) 是否一致	请求中的 prompt 前缀是否稳定一致
省掉成本	省字符串拼接	省重复 token 预处理，可能影响延迟/费用
重启后	失效	由服务端策略决定

key = json.dumps(context, sort_keys=True, ensure_ascii=False, default=str)

```
if key == _last_context_key and _last_prompt:
    return _last_prompt
_last_context_key = key
_last_prompt = assemble_system_prompt(context)
```

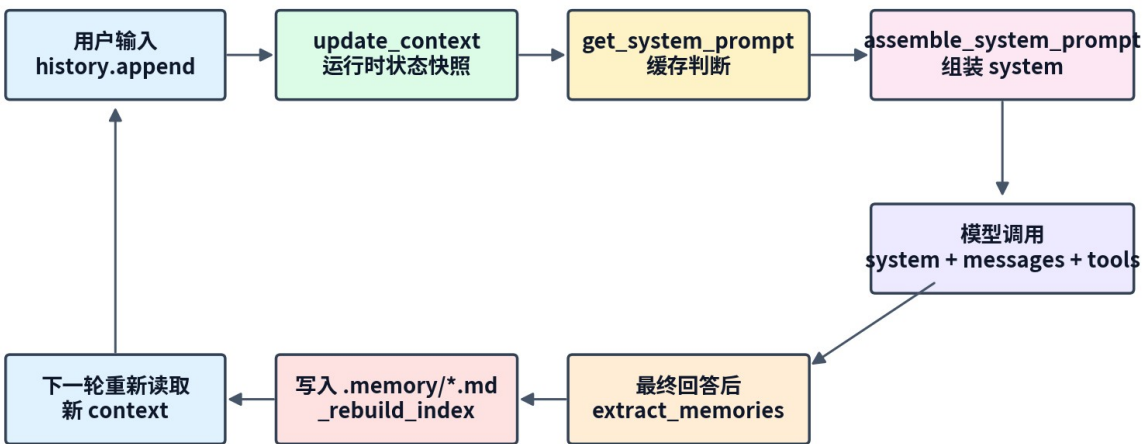
3. context / content / messages 的关系

对象	典型结构	职责
messages	[{"role": "user", "content": "..."}, ...]	保存真实对话历史和 tool_result。
content	单条 message 的正文	用户输入、assistant 回复、工具结果都放在 content 里。
context	{workspace, enabled_tools, skills, memory_index, memories}	当前运行状态快照，用来组装 system prompt。
system prompt	assemble_system_prompt(context) 的输出	交给模型的高级背景、规则、工具目录、memory、skills。

重点：context 不是 message；它不应该混入对话历史。messages 应尽量保持真实，避免调试时“历史里没有，实际请求里有”的隐形内容。

4. MEMORY.md 与完整 memory 全文注入

Memory / System Prompt 生命链路



核心原则：messages 只保存真实对话；memory 通过 context 注入 system prompt。

4.1 s09 参考答案的真实注入位置

重点：s10 参考答案没有注入“检索出的具体 memory 全文”；它只演示读取 MEMORY.md 内容进入 context。完整 memory 全文的检索和注入在 s09。

```
memories_content = load_memories(messages)
memory_turn = len(messages) - 1 if messages and isinstance(messages[-1].get("content"), str) else None

request_messages = messages.copy()
request_messages[memory_turn] = {
    **messages[memory_turn],
    "content": memories_content + "\n\n" + messages[memory_turn]["content"],
}
```

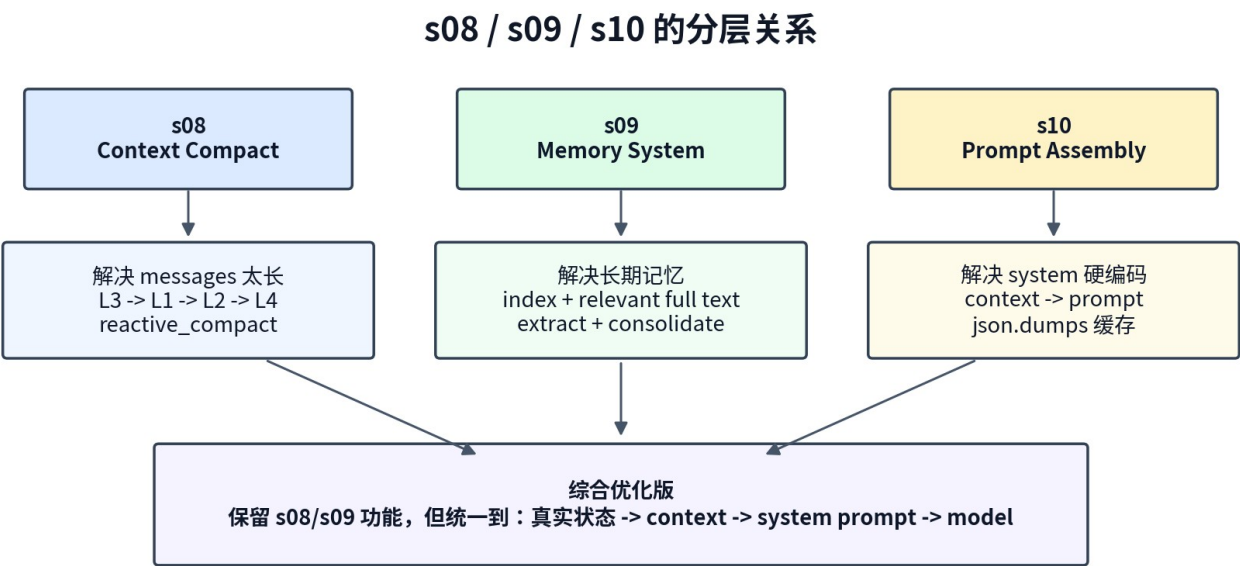
这段的含义：s09 把 <relevant_memories>...</relevant_memories> 临时拼到本轮用户问题前面，再发送给模型。

4.2 综合优化版的建议

设计	s09 教学版	综合优化版建议
MEMORY.md index	进 build_system(), 作为可用记忆目录	进入 context["memory_index"], 由 system prompt 渲染
相关 memory 全文	拼进当前 user.content 前面	进入 context["memories"], 由 system prompt 渲染
messages 是否被改写	请求时临时改写 request_messages	不改写；messages 保存真实对话
优点	实现快，适合教学 s09	边界清楚，适合 s08+s09+s10 综合版

重点：memory 写入发生在一轮结束后；memory 读取发生在下一轮开始前。新写入的 memory 通常不会影响当前这轮回答。

5. s08 / s09 / s10 参考答案的职责边界



版本	核心主题	它解决什么	不应该强行让它承担什么
s08	Context Compact	messages 太长: budget/snip/micro/summary/reactive	不负责 memory 检索和 prompt 动态组装
s09	Memory System	长期 memory 存储、选择、注入、提取、合并	不负责 runtime prompt caching
s10	System Prompt Assembly	根据 context 动态组装 system 并缓存	不完整演示 s09 的全文 memory 注入

6. 你的老代码对应问题与优化原因

位置	老代码 / 参考答案中的问题	优化版修改	为什么更好
tools 注入	PROMPT_SECTIONS["tools"] 写死 bash/read/write	从 context["enabled_tools"] 动态生成	真实工具和提示词一致，避免模型误判。
workspace 注入	workspace 在 PROMPT_SECTIONS 里固定	从 context["workspace"] 读取	context 才是唯一状态源。
memory 字段	context["memories"] 既可能是 index 又可能是全文	拆成 memory_index 和 memories	索引与全文语义分离。
memory 注入	system 里有索引，user.content 前又塞全文	全文也走 context -> system	messages 保持真实，压缩/提取更稳定。
compact 工具	可能先 compact 掉 assistant tool_use 再追加 tool_result	先返回 tool_result，完成配对后再 compact	符合工具调用协议。
unknown tool	只 print 然后 continue	返回错误 tool_result	每个 tool_use 都有对应 tool_result。
subagent	results 可能未定义	循环前 results=[]	避免第一次无工具调用就崩。
memory extraction	只用 pre_compress，可能漏最终 assistant	pre_compress + final assistant	可捕获最终结论和项目事实。
异常处理	except Exception: pass	打印 Memory selection/extraction failed	便于调试 JSON/API/编码错误。

重点：优化版不是比参考答案“教学上更好”，而是比直接把 s08/s09/s10 复制粘贴合体更适合真实综合代码。

7. Windows 编码报错复盘

UnicodeDecodeError: 'gbk' codec can't decode byte 0x94 in position 20

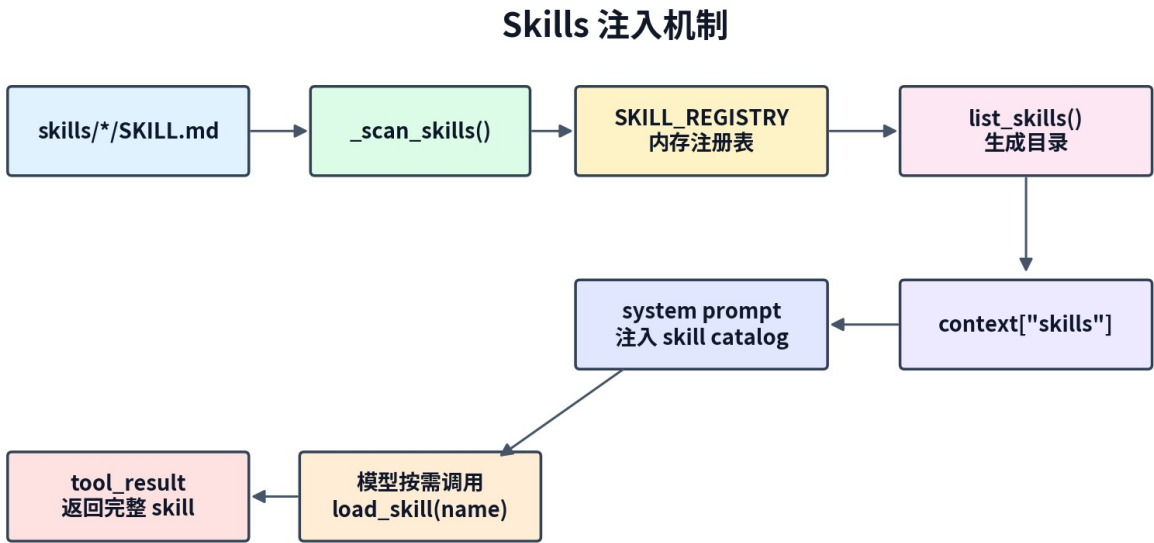
触发条件：write_file 成功写入 .memory/MEMORY.md，内容包含 UTF-8 字符 “—”；随后 update_context() 调 read_memory_index(), Path.read_text() 在 Windows 默认用 GBK 解码，于是失败。

问题点	错误写法	推荐写法
读取 MEMORY.md	MEMORY_INDEX.read_text()	MEMORY_INDEX.read_text(encoding="utf-8", errors="replace")
写入 memory 文件	filepath.write_text(text)	filepath.write_text(text, encoding="utf-8")
读取 skill 文件	manifest.read_text()	manifest.read_text(encoding="utf-8", errors="replace")

写 transcript	path.open("w")	path.open("w", encoding="utf-8")
--------------	----------------	----------------------------------

重点：凡是跨平台读写文本文件，尤其含中文、日文、emoji、长横线时，都应显式指定 UTF-8。

8. Skills 注入机制



system prompt 默认只注入目录；完整 skill 通过 load_skill 按需加载。

阶段	发生什么	注入内容
启动扫描	_scan_skills() 遍历 skills/*/SKILL.md	读取 name、description、content
注册表	写入 SKILL_REGISTRY	完整 skill 保存在内存
目录生成	list_skills()	只生成 name + description
context	update_context() 放入 context["skills"]	skill catalog
system prompt	assemble_system_prompt() 注入目录	告诉模型有哪些 skill
按需全文	模型调用 load_skill(name)	通过 tool_result 返回完整 SKILL.md

重点：skills 和 memory 很像，但方向不同：skill 全文通常由模型主动 load_skill；memory 全文通常由程序根据当前 messages 自动选择。

9. 最终推荐架构与检查清单

9.1 推荐架构

```

用户输入
-> history.append(user)
-> context = update_context(context, history)
-> system = get_system_prompt(context)
-> client.messages.create(system=system, messages=history, tools=TOOLS)
-> 工具循环 / 压缩 / 最终回答
-> extract_memories(...)
  
```

-> 写入 .memory/*.md + rebuild MEMORY.md
-> 下一轮重新 update_context

重点：一句话：真实状态进入 context；context 渲染 system；messages 保存真实对话；tool_use 必须配对 tool_result。

9.2 实现检查清单

- 每个 tool_use 都必须返回 tool_result，包括 unknown tool 和 permission denied。
- compact 不要破坏 assistant tool_use -> user tool_result 的相邻配对。
- 所有读写文本文件显式使用 encoding="utf-8"。
- 所有会影响 system prompt 的状态必须进入 context。
- 不要把 memory 拼进 user.content；通过 context -> system prompt 注入。
- context 变化后再调用 get_system_prompt，否则缓存可能返回旧 prompt。
- 异常不要全部 pass；至少打印调试信息。
- subagent 的临时变量提前初始化，避免未定义。
- skills 默认注入目录，完整内容按需 load_skill。

结束语：s10 的重点不是多写几个函数，而是把 prompt 当作 runtime state 的产物。